

A Computer Vision Approach for Parking Line Detections

Mohsin Muzammil, Abdus Shaikh

April 2024

Abstract

Accurate recognition of parking lines is crucial for autonomous parking systems. This paper presents a computer vision based approach to recognizing parking lines by utilizing an around-view monitoring (AVM) system. With real-time performance in mind, our method operates on two smaller regions of interest (ROI) within the surround view image to recognize a parking line near the vicinity of the vehicle. The ROIs are processed to remove noise before edge detection is performed. Due to the rectangular nature of parking lines, the edge gradient information is leveraged to extract the parking line endpoints. Finally, recognized lines are tracked using the vehicle odometry.

Our method emphasizes simplicity and effectiveness, streamlining the process of parking slot recognition without requiring complex sensor configurations. Experimental validation demonstrates the efficacy of our approach, achieving notable accuracy in parking line detection across various environmental conditions.

Introduction

As the automotive industry progresses towards autonomous driving capabilities, the development of reliable parking assistance systems remains a critical area of research. Automatic parking systems rely fundamentally on the accurate recognition of parking lines.

This paper proposes a computer vision-based approach for the recognition of parking lines within the context of autonomous parking systems. Our method utilizes an around-view monitoring (AVM) system which consists of cameras mounted on the front, rear, left, and right sides of the vehicle to show the driver a synthesized image of the top view of the vehicle. The advantages of working with AVM images is that only a single image needs be processed, the parking lines will be seen as straight lines, and a position in the top view image can be displayed as a relative distance from the vehicle which will be helpful for autonomous driving systems to path plan. To further improve processing time and to accurately detect parking lines near the vehicle, we operate on smaller regions of interest (ROI) within the AVM images located on the sides of the vehicle. It is difficult to recognize parking lines at a distance in the AVM images hence, recognized parking lines are then tracked using the vehicle odometry.

Common challenges when recognizing parking lines include pavement cracks, shadows, and variations in lighting and environments. To address pavement cracks and shadows, we utilize a crucial

observation: pixels corresponding to cracks and shadows typically exhibit lower intensities compared to the average pixel intensity in the image. Hence, our approach involves calculating the average pixel intensity of the AVM image and substituting dark pixels with the average pixel intensity value. This technique effectively eliminates pavement cracks and alleviates the impact of shadows. In tackling lighting and environmental variations, our strategy attempts to maximize the utilization of information from the original image post-preprocessing. We compute the edge magnitudes and directions through the application of the Sobel operator. By leveraging the rectangular nature of parking lines, we exploit edge gradient information to extract the endpoints of parking lines.

The significance of our approach lies in its simplicity, efficiency, and adaptability to diverse environmental conditions. By leveraging the inherent advantages of AVM systems and computer vision techniques, our method offers a promising solution for autonomous parking systems.

In the subsequent sections, we provide a detailed exposition of our proposed methodology, encompassing the various stages involved in parking line recognition and slot creation. Experimental validation and performance analyses further demonstrate the efficacy and reliability of our approach across different scenarios and environmental conditions.

Related Work

Various computer vision-based approaches have been proposed for recognizing parking lines in AVM images:

- Apply a line filter to isolate lines in the AVM image followed by a Hough Transform to extract the lines. Lastly, employ a deep learning method to check whether a corner on an extracted line is a parking line [2].
- Apply Canny Edge Detection combined with the Radon transform to detect parking lines in AVM images [4].
- Apply Sobel operator to find edges and their orientations, followed by the Random Sample Consensus (RANSAC) algorithm to find parallel line pairs. Additionally, vehicle odometry was used to track detected lines and improve robustness in various environments [3].

Our method will employ the Sobel operator to compute edge magnitudes and orientations. Then we apply a similar technique used in the Progressive Probabilistic Hough Transform [1] to extract parking line endpoints. We also track detected lines using the vehicle odometry.

The Method

In this section, we present the algorithm for recognizing and tracking parking.

Begin by creating a synthesized top view image of the vehicle by using the front, rear, left, and right cameras mounted on the vehicle. In our work, we have been working with AVM images of width 400 pixels and height 640 pixels.



Figure 1: AVM Image

The reliability of the AVM image diminishes as the distance from the vehicle increases due to occlusions and reduced pixel resolution. To address this, we operate on small regions of interest near the left and right cameras. Another benefit of this approach is increased computational efficiency. Rather than attempting to recognize parking lines in the entire AVM image, our method recognizes lines within these smaller regions and makes use of the vehicle movement to keep track of the recognized lines.

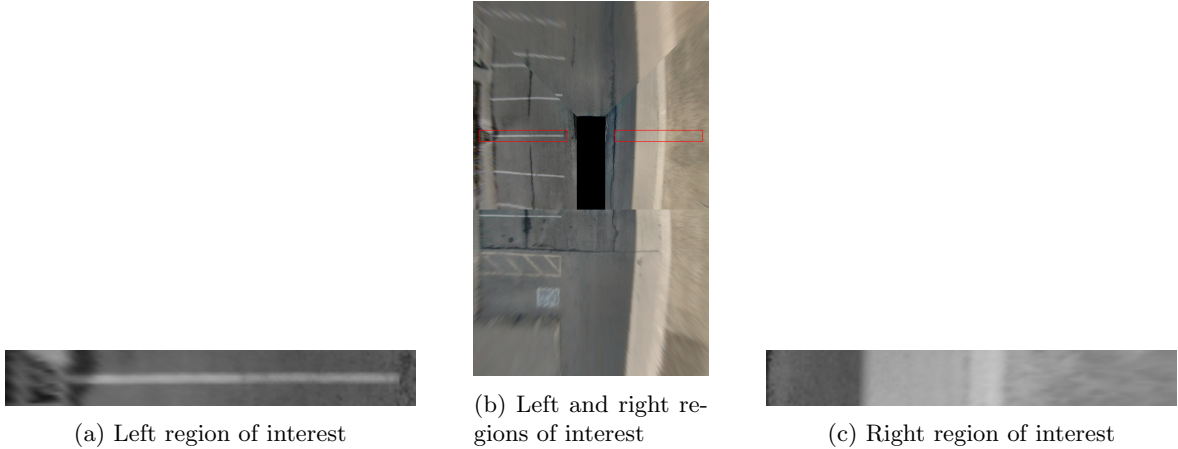


Figure 2: Extracted regions of interest from AVM image

The problem is now to recognize whether a parking line exists within the ROIs. To do this, we exploit the rectangular shape of parking lines by leveraging edge gradient information. However, pavement cracks and shadows may produce edge gradient information that makes it difficult to accurately extract the parking line endpoints. This is because pavement cracks and shadows will create noise in

the edge gradient information leading to obscured lines and false positives. To address this issue, we compute the average pixel intensity I_{avg} of the ROI and then replace all pixels in the ROI below I_{avg} by I_{avg} . This has an effect of removing cracks and lowering the impact of shadows on edge gradient information.

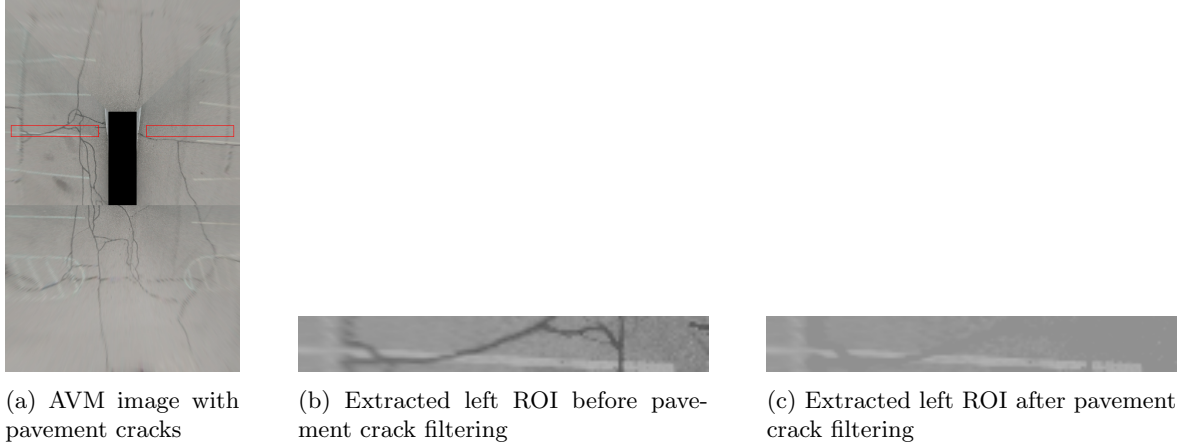


Figure 3: Pavement crack filtering

Next we smooth the processed image using a 3×3 Gaussian Blur (since our AVM images contained more salt and pepper noise we opted to use a 3×3 Median Blur).

Then we apply the Sobel Operator to find the magnitude and gradient direction of each.

$$G_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}, \theta = \arctan \left(\frac{G_y}{G_x} \right)$$

The angle is then rounded to one of four directions into one of four values.

- Edge gradients falling in range $[0^\circ, 22.5^\circ]$ or $[157.5^\circ, 180^\circ]$ maps to 0° .
- Edge gradients falling in range $(22.5^\circ, 67.5^\circ)$ maps to 45° .
- Edge gradients falling in range $[67.5^\circ, 112.5^\circ]$ maps to 90° .
- Edge gradients falling in range $(112.5^\circ, 157.5^\circ)$ maps to 135° .

Since parking lines have rectangular shape, the sides of the parking lines will emit edges with similar gradient directions. Next we filter the edge gradient information to suppress edges that most likely do not correspond to parking lines

- Suppress edges that are less than some percentage of the strongest edge
- Suppress vertical edges. Parking lines cannot appear vertical to the vehicle
- Suppress edges that are different from their immediate neighbours along the orthogonal gradient direction (i.e. edge direction). For example, for an edge whose gradient is vertical, compare the direction of the edge with its left and right immediate neighbours since the edge is horizontal.

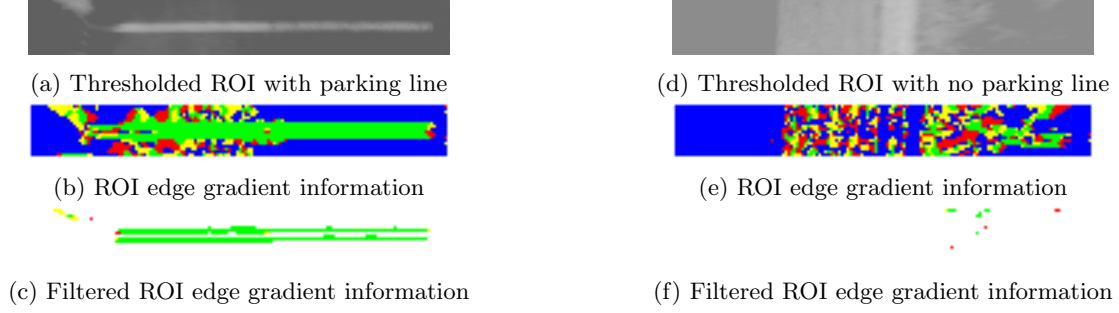


Figure 4: Each color corresponds to a rounded gradient angle; vertical gradients are colored green, horizontal gradients are blue, 45° gradients are red, and 135° gradients are yellow

This process results in a refined edge gradient map which best facilitates line search.

Now that we have the filtered edge gradient information, we can extract the parking line endpoints.

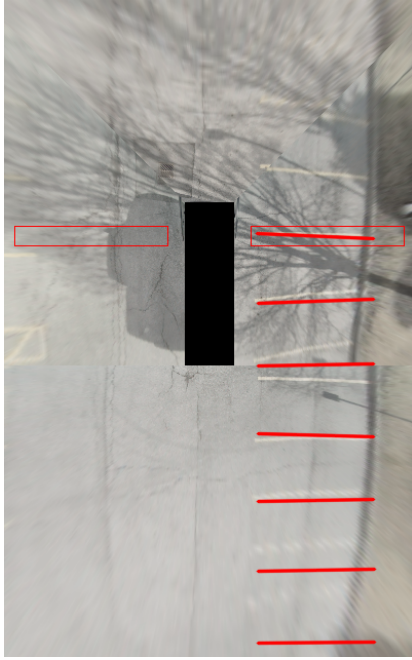
For each non-suppressed (remaining) edge e_{src}

- Starting at the location of e_{src} , travel in the orthogonal direction defined by the edge gradient of e_{src} . In other terms, travel in the direction of the edge. This will be one of the rounded angles $0^\circ, 45^\circ, 90^\circ$, or 135° which corresponds to one of e_{src} 's immediate neighbours.
- If we arrive at an edge that shares the same direction as e_{src} , we know we are continuing along the same line. Update the endpoints of the line and continue to the next iteration.
- If we arrive at an edge that has a different edge direction as e_{src} or is suppressed, we have exited the line and must attempt to find the next edge that will continue this line. To do this we continue searching along the same direction while incrementing a "gap" variable which will define a limit for how far to continue searching. To increase our chances of finding an edge that will continue the line, we also test the edges in the directions $e_{src.dir} + 45^\circ$ and $e_{src.dir} + 315^\circ$. If we find an edge to continue the line such that it shares the same direction as e_{src} before surpassing the pre-defined gap limit, update the line endpoints and continue to the next iteration. Otherwise terminate search.
- Then we do the same process by starting at e_{src} , but this time travelling in the opposite direction.

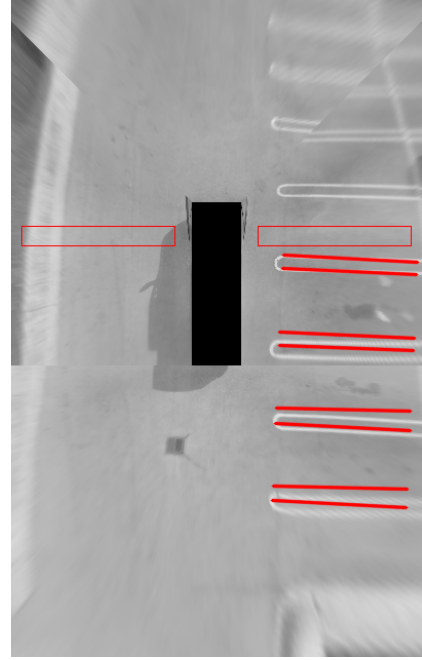
Once the maximum possible line is extracted, we check if the line length meets a minimum requirement. If so, return the endpoints of that line.

Next, we make use of a data structure to keep track of N left and right detected parking lines. In our implementation, we created a basic circular buffer class and instantiated it twice, one for left parking lines and one for the right parking lines. Once the endpoints of a line meeting the minimum line length is extracted. We insert it into the respective circular buffer. Each frame, we update the tracked lines using the vehicle's change in horizontal Δx , vertical Δy , angle $\Delta \theta$ movement. Moreover, the circular buffers keep track of some basic statistics about its tracked lines. We kept track of the average start point, end point and line direction vector in order to complete partial detections and maintain uniformity amongst the lines. For instance, line occlusions may result in partial detections, we can then use the circular buffer to extend the line endpoints to the average start and end points.

The circular buffers can also help with filtering false positives by maintaining a minimum distance between lines.



(a) AVM Image with parking line detection and tracking



(b) AVM Image with parking line detection and tracking

Figure 5: Parking line detection and tracking

Results

References

- [1] C. Galamhos, J. Matas, and J. Kittler. “Progressive probabilistic Hough transform for line detection”. In: (1999), pp. 554–560. DOI: 10.1109/CVPR.1999.786993.
- [2] Young Lee and Minji Park. “Around-View-Monitoring-Based Automatic Parking System Using Parking Line Detection”. In: *Applied Sciences* 11 (2021), p. 11905. DOI: 10.3390/app112411905. URL: <https://doi.org/10.3390/app112411905>.
- [3] Jae Kyu Suhr and Ho Gi Jung. “A Universal Vacant Parking Slot Recognition System Using Sensors Mounted on Off-the-Shelf Vehicles”. In: *Sensors* 18.4 (2018), p. 1213. ISSN: 1424-8220. DOI: 10.3390/s18041213. URL: <https://www.mdpi.com/1424-8220/18/4/1213>.
- [4] Cheng Wang et al. “Automatic Parking Based on a Bird’s Eye View Vision System”. In: *Advances in Mechanical Engineering* 6 (2014). DOI: 10.1155/2014/847406.